

UART FIFO Mini Project

UART Transmitter, Receiver & FIFO Integration Using Verilog HDL

This mini-project implements a complete UART communication system using Verilog HDL. The design integrates three major modules: UART Receiver (RX), FIFO Buffer (8x8), and UART Transmitter (TX) coordinated by a Top Controller FSM. Serial data is received bit-by-bit, assembled into parallel bytes, stored in the FIFO, and then retransmitted serially. The design is fully synchronous and verified using ModelSim waveform simulation.

Project Objectives

- Design and implement UART Receiver (RX) with start/data/stop bit detection
- Design and implement UART Transmitter (TX) with LSB-first serial output
- Implement an 8-location x 8-bit synchronous FIFO buffer with full/empty flags
- Integrate RX → FIFO → TX data path with a controller FSM
- Verify end-to-end operation using ModelSim simulation waveforms

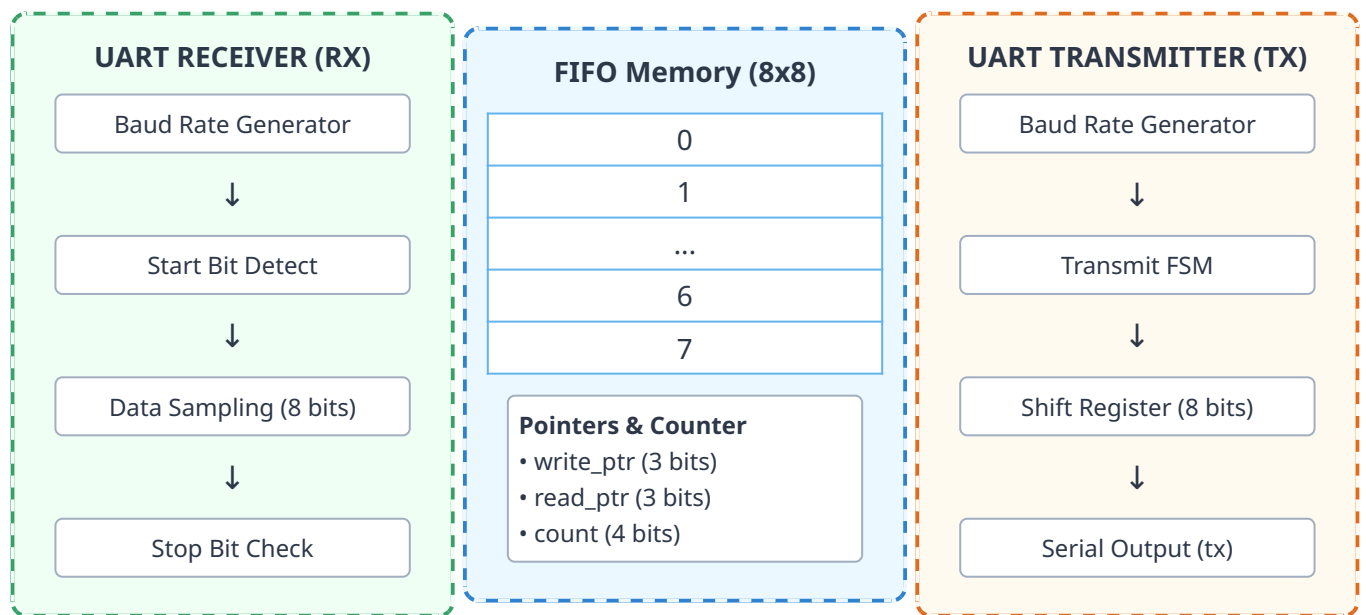
1. Overall System Block Diagram



Signal Description

Signal	Direction	Width	Description
clk	Input	1	System clock - drives all synchronous logic
rst	Input	1	Active-high synchronous reset
rx_data[7:0]	RX → FIFO	8	Parallel byte produced by UART Receiver
rx_done	RX → FIFO	1	Pulse asserted when a byte is ready
fifo_data[7:0]	FIFO → TX	8	Byte read from FIFO to feed the Transmitter
fifo_empty	FIFO → TX	1	High when FIFO holds no valid data
tx_start	Ctrl → TX	1	Pulse that begins one UART transmission
tx_done	TX → Ctrl	1	Pulse asserted when transmission is complete

2. Architecture Diagram - Data Flow



How Data Flows Through the System

When a UART frame arrives on the RX line, the Baud Rate Generator divides the system clock to produce a 16x oversampling tick. The Start Bit Detector watches for a high-to-low transition indicating the start bit. Once detected, the Data Sampling block captures all 8 data bits at the centre of each bit period to maximise noise immunity. The Stop Bit Check verifies the line returns high; if valid, rx_done is asserted for one clock cycle along with the 8-bit parallel data on rx_data[7:0].

The FIFO buffer receives the byte on the rising edge of wr_en (which is driven by rx_done). The FIFO maintains a write_ptr, a read_ptr, and a 4-bit count register. The full flag prevents overflow; the empty flag tells the controller when no data is waiting.

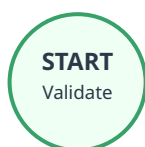
On the transmit side, the Top Controller FSM reads the FIFO whenever it is not empty, presents the data to the TX module, asserts tx_start, and waits for tx_done before returning to idle. The TX Transmit FSM serialises the byte LSB-first through the 8-bit Shift Register, framing it with start and stop bits.

3. FSM State Diagrams

1. UART RX FSM



↓ rx=0 (start)



↓ valid start

2. UART TX FSM



↓ tx_start=1



↓

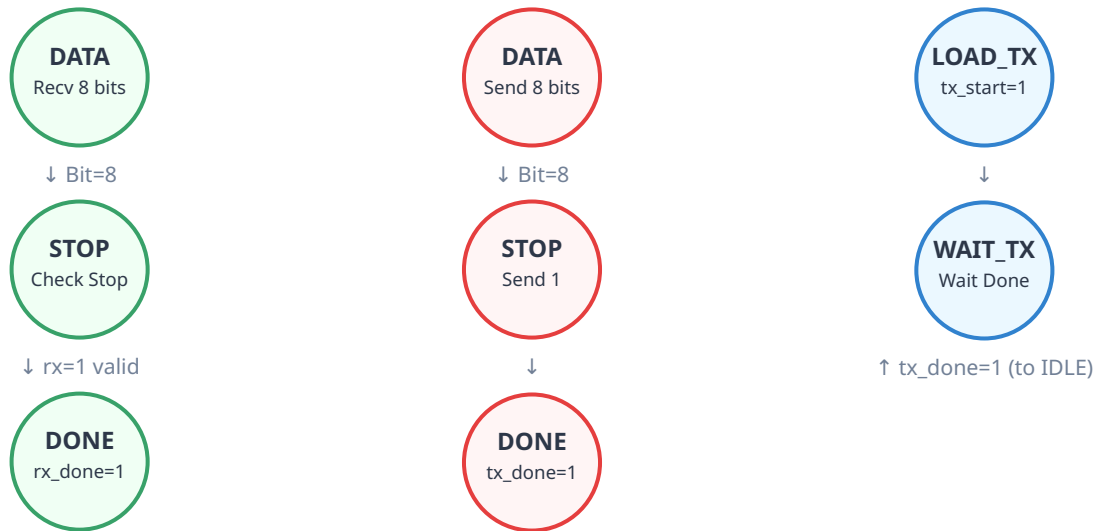
3. Top Controller FSM



↓ fifo_empty=0



↓



State Encoding: IDLE=00, READ_FIFO=01, LOAD_TX=10, WAIT_TX=11

FSM State Descriptions

3a. UART Receiver FSM

State	Condition to Enter	Action / Output
IDLE	Default/reset	Wait; monitor rx line for falling edge
START	rx=0 detected	Wait half-bit period to centre; validate start bit
DATA	Start bit confirmed	Sample rx at bit centre; store into shift reg; repeat 8x
STOP	All 8 bits received	Verify rx=1 (stop bit); flag error if rx=0
DONE	Stop bit valid (rx=1)	Assert rx_done=1 for one clock; write to FIFO

3b. UART Transmitter FSM

State	Condition to Enter	Action / Output
IDLE	Default/reset / tx_start=0	Hold tx=1 (line idle); wait for tx_start
START	tx_start=1	Drive tx=0 for one full bit period
DATA	Start bit sent	Shift out 8 bits LSB-first; one bit per baud period
STOP	8 bits transmitted	Drive tx=1 for one full bit period
DONE	Stop bit sent	Assert tx_done=1 for one clock; return to IDLE

3c. Top Controller FSM

State (Code)	Condition to Enter	Action
IDLE (00)	Reset / fifo_empty=1 / tx_done=1	Poll FIFO empty flag; stay idle when FIFO empty
READ_FIFO (01)	fifo_empty=0	Assert rd_en=1 for one clock to pop byte from FIFO
LOAD_TX (10)	rd_en pulse complete	Assert tx_start=1 for one clock; latch fifo_data
WAIT_TX (11)	tx_start pulse sent	Hold; wait until TX asserts tx_done; then → IDLE

4. FIFO Operation - Example with 3 Bytes

The FIFO is implemented as a circular buffer with a synchronous write port and a synchronous read port. Three 3-bit pointers (write_ptr and read_ptr) index into an 8-entry x 8-bit register array. A 4-bit count register tracks occupancy and drives the full (count=8) and empty (count=0) status flags. The table below traces the pointer and counter changes as bytes 0x53, 0xA5, and 0x11 are written then read back in FIFO order.

Step	Operation	write_ptr	read_ptr	count	FIFO Content
1	Reset	000	000	0	[_ _ _ _ _ _ _ _]
2	Write 53	001	000	1	[53, _ _ _ _ _ _ _]
3	Write A5	010	000	2	[53, A5, _ _ _ _ _ _]
4	Write 11	011	000	3	[53, A5, 11, _ _ _ _ _]
5	Read 53	011	001	2	[53, A5, 11, _ _ _ _ _]
6	Read A5	011	010	1	[53, A5, 11, _ _ _ _ _]
7	Read 11	011	011	0	[53, A5, 11, _ _ _ _ _]

Key FIFO Design Rules

- **Write:** data[write_ptr] = din; write_ptr = write_ptr + 1; count = count + 1 (only when wr_en=1 and full=0)
- **Read:** dout = data[read_ptr]; read_ptr = read_ptr + 1; count = count - 1 (only when rd_en=1 and empty=0)
- **Full flag:** count=8 → prevent further writes to avoid data corruption
- **Empty flag:** count=0 → prevent spurious reads; controller stays in IDLE
- **Circular wrap:** pointers use modulo-8 arithmetic; after reaching 7 they wrap to 0

5. Verilog Module Overview

Module	Key Parameters	Inputs	Outputs
uart_rx	CLK_HZ/BAUD, BAUD_DIV	clk, rst, rx	rx_data[7:0], rx_done
uart_tx	CLK_HZ/BAUD, BAUD_DIV	clk, rst, tx_data[7:0], tx_start	tx, tx_done
fifo_8x8	DEPTH=8, WIDTH=8	clk, rst, wr_en, rd_en, din[7:0]	dout[7:0], full, empty
top_controller	-	clk, rst, rx_done, tx_done, fifo_empty	wr_en, rd_en, tx_start
uart_top	-	clk, rst, rx	tx

6. Simulation & Waveform Verification

Functional verification is performed in ModelSim by applying a serial UART frame for each test byte (0x53, 0xA5, 0x11) at the RX input and observing all internal and output signals. The testbench drives rx with correctly timed start, data, and stop bits at the chosen baud rate. The key waveform checkpoints are listed below.

Checkpoint	Expected Observation
RX Frame	rx transitions low (start bit) → 8 data bits → high (stop bit)
rx_done pulse	Single-clock pulse appears after the stop bit is validated
FIFO write	count increments by 1; write_ptr advances; fifo_empty de-asserts
Controller	Transitions IDLE → READ_FIFO → LOAD_TX → WAIT_TX → IDLE
tx_start pulse	Single-clock pulse issued by controller to begin TX
TX Frame	tx goes low (start) → 8 bits LSB-first → high (stop)
tx_done pulse	Single-clock pulse after stop bit; FIFO count decrements

7. Applications & Future Scope

Applications

- FPGA-based serial communication bridges
- Embedded microcontroller debug interfaces
- Industrial sensor data acquisition links
- PC ↔ FPGA loopback test systems
- Serial console for SoC designs
- Data logging and telemetry modules

Future Enhancements

- Configurable baud rate via register
- Optional parity bit (even / odd)
- Larger FIFO depth (16, 32, 64 entries)
- Interrupt-driven RX/TX operation
- AXI-Lite / APB bus interface wrapper
- Hardware flow control (RTS/CTS)

8. Conclusion

A fully functional UART communication sub-system was successfully designed in Verilog HDL and verified through simulation. The modular approach separating receiver, FIFO, transmitter, and controller into distinct RTL blocks ensures clean interfaces, easy reuse, and straightforward FPGA synthesis. The 8x8 FIFO effectively decouples the RX and TX baud-rate domains, preventing data loss when back-to-back bytes arrive faster than they can be re-transmitted. The Top Controller FSM provides deterministic, glitch-free handshaking between all blocks. This project forms a solid foundation for more advanced serial communication IP cores targeting FPGA and ASIC implementations.